TP 5G sur labtainer

KHADIM BA, PH. JAILLON

Avril 2026

1 Introduction

Ce TP a pour objectif de vous familiariser avec le fonctionnement d'un réseau 5G, en particulier avec le processus d'enregistrement d'un terminal utilisateur (UE), depuis son attachement au réseau jusqu'à l'établissement d'une session de données.

Les composants du réseau 5G (cœur de réseau Open5GS, gNB et UE simulés via UERANSIM) sont déjà préinstallés dans l'environnement Labtainer. Vous n'aurez donc pas à déployer l'infrastructure, mais devrez vous concentrer sur :

— le lancement des différents services du réseau,
— l'enregistrement d'un terminal UE dans le réseau 5G,
— l'analyse des échanges de signalisation et des flux de données à l'aide de Wireshark.

Ce TP vous permettra de visualiser concrètement les étapes clés du fonctionnement d'un réseau 5G, de comprendre les messages échangés entre les différentes entités du réseau, et de développer une première expérience d'analyse du trafic dans une architecture 5G.

2 Architecture du Lab

Dans ce TP, l'architecture du réseau 5G est virtualisée sous forme de conteneurs déployés dans l'environnement Labtainer. Chaque fonction du réseau est isolée dans un conteneur représentant un composant spécifique de l'architecture 5G.

— **5G Core** : Le cœur de réseau est implémenté avec *Open5GS*. Les différentes fonctions du cœur (AMF, SMF1, SMF2, UPF1, UPF2, AUSF, UDM, UDR, PCF, BSF, SCP, NSSF et NRF) sont réparties sur plusieurs conteneurs interconnectés via le réseau *100.0.0.0/24*. L'architecture utilisée est de type *Service-Based Architecture (SBA)*.

  Deux plans de données sont configurés afin d'illustrer un scénario combinant **Cloud et Edge**. Les fonctions *SMF1* et *UPF1* représentent le traitement centralisé dans le cloud, tandis que *SMF2* et *UPF2* sont positionnés plus près de l'accès radio afin de simuler un déploiement **Edge**.

— **5G RAN** : Côté accès radio, un conteneur *gNB* (Next Generation NodeB) est utilisé pour établir la connexion avec les terminaux utilisateurs.

  Ce conteneur possède deux interfaces réseau :

  — une interface connectée au réseau du cœur 5G (*172.22.0.0/24*),
  — une interface connectée au réseau d'accès radio (*172.25.0.0/24*).

  Deux conteneurs supplémentaires, *ue1* et *ue2*, simulent des terminaux utilisateurs grâce à l'outil *UERANSIM*, qui permet d'émuler le comportement d'un terminal 5G.

  Ces deux terminaux communiquent avec le gNB pour la signalisation (messages NAS et RRC) et établissent des sessions de données vers différents réseaux de données (DNN) :

  — **UE1** est associé au **DNN Cloud** et utilise le réseau utilisateur *10.0.100.0/24*.
  — **UE2** est associé au **DNN Edge** et utilise le réseau utilisateur *10.0.101.0/24*.

— **Edge Server** : Un conteneur supplémentaire représente un serveur applicatif déployé à l'Edge du réseau, connecté au plan utilisateur via *UPF2*. Ce serveur est accessible depuis les terminaux via le réseau de données Edge et permet d'illustrer le principe de *Multi-access Edge Computing (MEC)*.

Dans ce TP, ce serveur est utilisé pour tester la connectivité et comparer les performances (latence et temps de réponse) entre les services hébergés au **Cloud** et ceux hébergés à l'**Edge**.

— **Subscriber** : Un conteneur supplémentaire est utilisé pour gérer les abonnés du réseau. Il regroupe :

— une base de données *MongoDB* qui stocke les informations des abonnés (IMSI, clés d'authentification, profils de connectivité),

— une interface Web (*WebUI*) permettant d'ajouter et de gérer les abonnés via un navigateur.

Les abonnés correspondant aux terminaux *ue1* et *ue2* sont déjà préconfigurés dans cette base de données.

— **tap et netmon** : deux conteneurs supplémentaires ont été ajoutés dans ce lab afin de permettre l'observation et l'analyse du trafic réseau du cœur 5G.

— Conteneur `tap` est utilisé pour capturer le trafic réseau circulant entre les différentes fonctions du cœur 5G. Il agit comme un point d'écoute (sniffer) et enregistre les échanges dans des fichiers de capture (format `.pcap`).

— Conteneur `netmon` permet d'analyser ces captures. Il contient notamment Wireshark ainsi que les plugins nécessaires pour décoder les échanges SBI (OpenAPI).

Ces deux conteneurs permettent ainsi de séparer la capture du trafic (tap) de son analyse (netmon), facilitant l'observation du fonctionnement interne du cœur 5G.

L'architecture du déploiement est décrite dans la figure suivante :

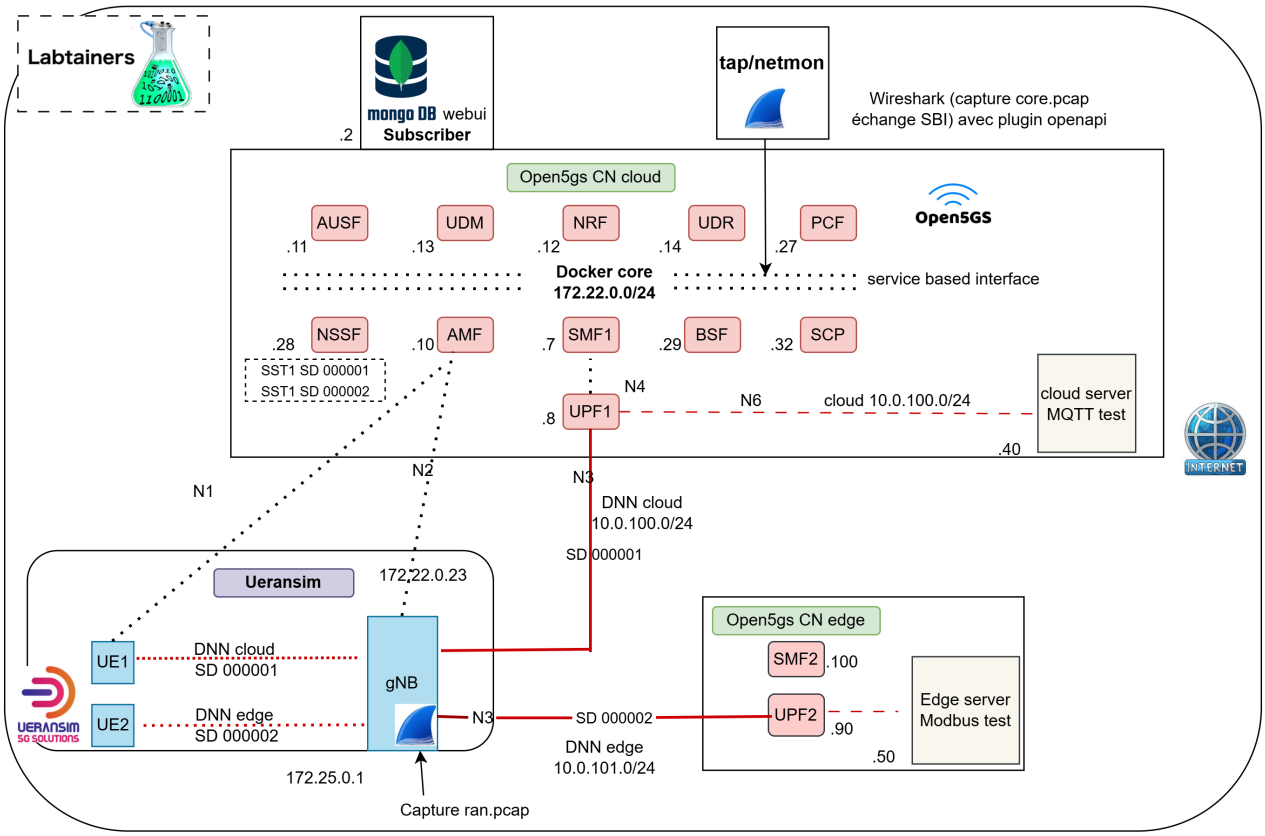


FIGURE 1 – Architecture du réseau 5G sur Labtainer

3 Étapes à suivre

3.1 Lancement du lab

Vous recevez le lab sous la forme d'une archive :
`5g-lab2.tgz`
Cette archive doit être copiée dans le répertoire des laboratoires de Labtainer puis décompressée :

```
cd ~/labtainer/trunk/labs/
tar -xzf 5g-lab2.tgz
```

Ensuite, se placer dans le répertoire étudiant :

```
cd ~/labtainer/labtainer-student
```

Puis reconstruire et lancer le lab avec la commande :

```
rebuild.py -b 5g-lab2
```

Cette commande construit les conteneurs du lab.
Pour démarrer le lab :

```
labtainer 5g-lab2
```

Après le lancement du lab, deux terminaux principaux s'ouvrent automatiquement.
Terminal Core Network
Ce terminal regroupe les conteneurs liés au cœur de réseau et aux serveurs applicatifs :
— upf1
— upf2
— edgeserver
— cloudserver
— netmon (capture sbi)
Terminal RAN
Ce terminal regroupe les conteneurs du réseau d'accès radio :
— gnb
— ue1
— ue2
Afin de simplifier l'affichage pour les étudiants, seuls les conteneurs nécessaires à la réalisation du TP sont affichés dans ces terminaux.

3.2 Démarrage des équipements utilisateurs (UE)

Le gnb et les terminaux utilisateurs doivent être lancés manuellement.
Sur le conteneur `gnb`, exécuter :

```
start_gnb
```

Démarrer directement wireshark sur le gnb pour l'analyse des échanges :

```
wireshark -k -i any
```

Sur le conteneur `ue1`, exécuter :

`start_ue1`

Sur le conteneur `ue2`, exécuter :

`start_ue2`

Les logs affichent alors la procédure d'enregistrement au réseau 5G ainsi que l'établissement de la session PDU. Vous pouvez noter les adresses IP de ue1 et ue2.

Pour arrêter ou deconnecter les UE (à la fin du tp si besoin) :

```
stop_ue1
stop_ue2
```

3.3 Vérification de la connectivité

Une fois les UE connectés, vérifier la connectivité réseau.

Depuis `ue1` :

`ping 10.0.100.1`

Depuis `ue2` :

`ping 10.0.101.1`

Ces adresses correspondent aux passerelles des réseaux associés respectivement au DNN Cloud et au DNN Edge.

Tester également la connectivité inverse entre les réseaux afin de vérifier le bon fonctionnement du routage.

3.4 Comparaison des latences Edge et Cloud

Afin d'observer l'impact du placement des applications, mesurer la latence entre les serveurs et les terminaux utilisateurs.

Depuis le conteneur `upf1`, mesurer la latence vers `ue1` :

`ping <IP_UE1>`

Depuis le conteneur `upf2`, mesurer la latence vers `ue2` :

`ping <IP_UE2>`

Comparer les temps de latence observés entre l'architecture Edge et Cloud.

3.5 Test d'une application IoT avec MQTT

Dans ce scénario, `ue1` agit comme client IoT et communique avec une application hébergée sur le serveur Cloud via le protocole MQTT.

A faire :

- lancer le broker MQTT sur le `cloudserver` :
  ```
  mosquitto_sub -t "test" -v
  ```
- publier un message depuis `ue1` vers `cloudserver` (ip 172.22.0.40) en utilisant `mosquitto_pub`
- vérifier la réception du message sur le `cloudserver`
- analyser les échanges MQTT dans Wireshark avec le filtre :
  ```
  mqtt
  ```
- identifier :
  - les adresses IP source/destination
  - le topic MQTT utilisé
  - les échanges Publish/Subscribe

3.6 Test d'une application industrielle avec Modbus

Dans ce scénario, `ue2` simule un équipement industriel (PLC, automate etc...) communiquant avec une application déployée sur le serveur Edge (contrôle/lit).

Vous devez :

— lancer l'application Modbus sur `ue2` :
```
start_plc
```
— envoyer des données depuis `edgeserver` :
```
python3 application.py
```
— vérifier la réception et le traitement des données sur le serveur Edge
— analyser les échanges Modbus TCP dans Wireshark avec le filtre :
```
modbus
```
— identifier :
  — les fonctions Modbus utilisées
  — les requêtes et réponses échangées
  — les éventuelles exceptions Modbus

3.7 Analyse des échanges SBI

L'objectif de cette partie est d'analyser les échanges entre les fonctions du cœur 5G à travers l'interface SBI à l'aide de Wireshark.

— se placer sur le terminal `netmon` :
— se placer dans le répertoire contenant les captures :
`cd /taps`
— ouvrir une capture avec Wireshark :
`wireshark core.pcap`
— appliquer le filtre suivant dans Wireshark :
`openapi`

Travail demandé

Dans le fichier `core.pcap`, sélectionner et analyser un exemple pour chacun des échanges suivants :

— `RegisterNFInstance`
— `CreateSubscription`
— `UpdateNFInstance`

Pour chaque échange, répondre aux questions suivantes :

1. **Identification de l'échange**
   — Quelle est la fonction source ?
   — Quelle est la fonction destination ?
   — Quel est le type d'opération ?
2. **Analyse de la requête (Request)**
   — Quelle est la méthode HTTP utilisée ?
   — Quelle est l'URI ?
3. **Analyse de la réponse (Response)**
   — Quel est le code de réponse HTTP ?
4. **Analyse du contenu SBI (HTTP/2 - OpenAPI)**
   — Effectuer un `Follow HTTP/2 Stream` sur un échange SBI.
   — Observer le contenu JSON échangé entre les NF.
   — Identifier les informations de la NF :

— identité de la NF (`nfInstanceId`)
— type de NF (`nfType`)
— état de la NF (`nfStatus`)
— services exposés (`nfServiceList`)
— adresse IP et ports des services